SYSTEMS ENGINEERING WHITEPAPER

PulseQueue: Distributed Background Task Queue & Job Engine

System Architecture, Failure-Domain Mitigation, and Under-the-Hood Algorithms

Author: Abir (pulse-queue)

Repository: github.com/abirc6733/pulse-queue

Target Stack: Go, Redis, Docker

Benchmarked Throughput: >112,000 ops/sec

1. What Are We Solving? (The Problem Space)

1.1 The Synchronous Request-Response Anti-Pattern

In modern web applications, user-facing endpoints frequently trigger ancillary operations: payment settlements, thumbnail generations, analytical event dispatches, PDF invoice renderings, and transactional emails. When software architectures attempt to execute these operations synchronously inside the incoming HTTP request thread, system performance degrades catastrophically:

- **Client Latency Degradation:** A user waiting for a checkout confirmation is blocked for 5–15 seconds while external third-party APIs (e.g., Stripe, SendGrid) respond.
- **Cascading Outages:** If an external email provider suffers a 503 outage or latency spike, upstream web worker threads are held open, rapidly exhausting the web server's connection pool.
- **Flash Crowd Vulnerability:** During high-concurrency events (e.g., flash sales, Black Friday), executing CPU-intensive tasks inline causes out-of-memory (OOM) kernel panics.

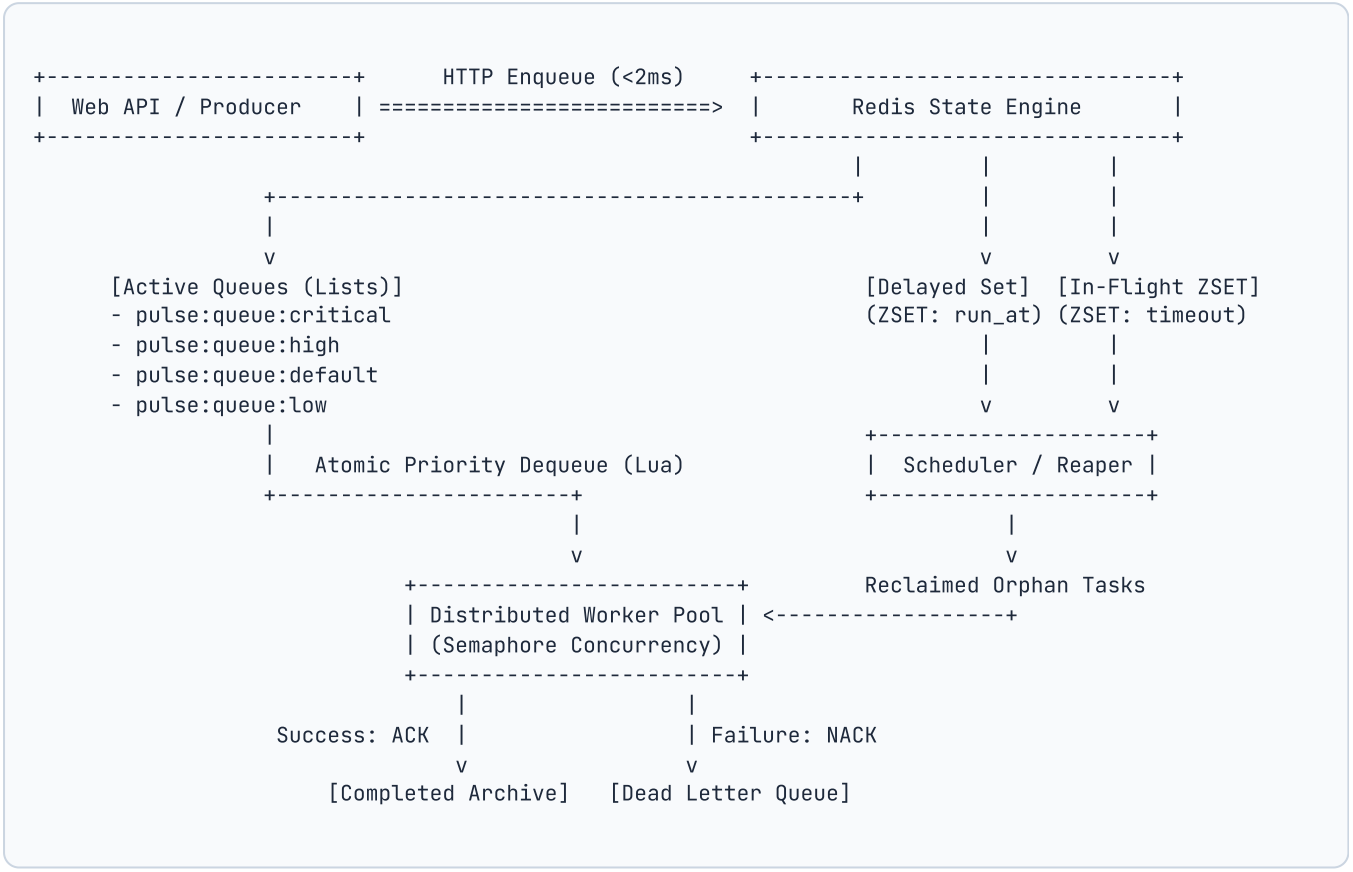
1.2 The Fatal Flaws of Naive In-Memory Solutions (Goroutines / Threads)

A frequent beginner approach is simply spawning unmanaged background threads (e.g., `go sendEmail()`). In production, this creates catastrophic vulnerabilities:

> **Why In-Memory Background Tasks Fail in Production:**
>
> - **Process Restarts = Irrecoverable Data Loss:** If a Kubernetes pod restarts, deploys a new image, or gets OOM-killed, all jobs queued in server RAM vanish immediately without an audit trail.
> - **Lack of Distributed Visibility:** In-memory queues cannot be shared across 20 horizontal microservice instances. One overloaded pod has no way to offload work to an idle pod.
> - **Double-Spend / Duplicate Execution:** Double-clicking a button triggers two parallel routines without cluster-wide deduplication or locking.
> - **Absence of Backpressure & Throttling:** Spawning 100,000 goroutines concurrently exhausts database connection pools and file descriptors.

2. How PulseQueue Solves It (Architectural Blueprint)

PulseQueue introduces an asynchronous, decoupled, multi-node architecture utilizing **Redis** as a persistent, high-throughput atomic state engine combined with a distributed fleet of **Go worker nodes**.



3. The Algorithms Behind PulseQueue

3.1 Algorithm 1: Atomic State Transitions via Redis Lua Scripts

The Problem: When 50 worker nodes concurrently pop tasks from a priority queue and record in-flight tracking, traditional Redis commands require multi-step round-trips. Standard `WATCH/MULTI/EXEC` uses Optimistic Concurrency Control (OCC), leading to massive transaction abort rates under load.

The Solution: PulseQueue executes all state transitions using single-threaded, embedded **Redis Lua scripts**. Lua scripts execute atomically inside Redis memory with zero possibility of interleaving operations.

```
-- Dequeue Algorithm with Strict Priority Arbitration
for i = 1, 4 do
    local q_key = KEYS[i] -- critical, high, default, low
    local task_id = redis.call("RPOP", q_key)
    if task_id then
        -- Atomically register into processing ZSET with visibility deadline
        redis.call("ZADD", KEYS[5], ARGV[2], task_id)
        return task_id
    end
end
return nil
```

3.2 Algorithm 2: At-Least-Once Delivery & Visibility Timeout Reaper

To solve the classic distributed systems challenge where a worker crashes mid-execution (power failure, `kill -9`, or network partition), PulseQueue implements a **Visibility Timeout** model:

- When a worker picks up a task, the task is *not removed from the system*. It is placed in a sorted set (`pulse:processing`) with its score set to:

```
Score = UnixTimestamp(Now) + TaskTimeout
```

- **Happy Path:** The worker finishes execution and sends an atomic `ACK` script. Redis removes the task from `pulse:processing` and marks it `completed`.
- **Crash Path:** If the worker crashes, the background **Reaper Daemon** queries Redis for all tasks whose visibility deadline has expired:

```
ExpiredTasks = ZRANGEBYSCORE(pulse:processing, 0, CurrentTimestamp, LIMIT 0,
                                                                    BatchSize)
```

The reaper extracts the expired task and re-queues it back to its original priority lane or triggers an automated retry. **No task is ever permanently lost.**

3.3 Algorithm 3: Exponential Backoff with Jitter

When an external dependency fails, immediately retrying all failed tasks results in the **Thundering Herd Problem**, worsening the target outage. PulseQueue applies truncated exponential backoff combined with randomized full jitter:

$$t_{retry} = 2^{retry_count} \times t_{base} + \text{rand}(0, 0.5 \times 2^{retry_count})$$

The calculated retry timestamp is scored into the delayed sorted set (`pulse:delayed`). The **Scheduler Daemon** periodically polls the delayed set and migrates ready tasks to active queues as soon as their target run timestamp arrives.

3.4 Algorithm 4: Distributed Idempotency (Deduplication)

PulseQueue guarantees idempotency through atomic fingerprinting. Producers can supply a `UniqueKey` (e.g. `invoice-charge-9921`) with a Time-To-Live (TTL):

```
-- Atomic Idempotency Check in Enqueue Script
if KEYS[4] ~= "" then
    if redis.call("EXISTS", KEYS[4]) == 1 then
        return 0 -- Duplicate rejected immediately
    end
    redis.call("SET", KEYS[4], ARGV[1], "EX", tonumber(ARGV[5]))
end
```

4. Concurrency Management & Resilience

4.1 Semaphore-Bounded Goroutine Pooling

Unlike naive Go implementations that launch unbounded goroutines, PulseQueue constrains memory consumption using a **buffered channel semaphore**:

```
// Semaphore concurrency gate
semaphore := make(chan struct{}, config.Concurrency)

// Inside worker loop:
semaphore ← struct{}{} // Acquire slot
go func(task *types.Task) {
    defer func() {
        ←semaphore    // Release slot
        wg.Done()
    }()
    s.executeTask(ctx, task)
}(task)
```

4.2 Panic Recovery Safeguard

A panic in user-provided business logic (e.g. nil pointer dereference) would ordinarily crash the entire Go runtime process. PulseQueue isolates every task execution inside a deferred panic interceptor:

```
defer func() {
    if r := recover(); r ≠ nil {
        execErr = fmt.Errorf("task panicked: %v", r)
    }
}()
```

If a panic occurs, the worker engine converts the panic into a structured error, reports it to Redis, routes the task towards retry/DLQ, and continues processing other queue items uninterrupted.

5. Dead Letter Queue (DLQ) & Operator Management

Tasks that exhaust their maximum configured retry threshold (e.g., 3 retries) are designated as `StateDead` and pushed into the Dead Letter Queue (`pulse:dlq`). This isolates poison-pill payloads from degrading operational throughput while maintaining full debugging visibility:

Component	Purpose	Operator Action
DLQ Inspector	Stores permanently failed tasks along with their stack trace and last error.	Inspect payload and root-cause failure via Web UI or CLI.
Atomic Re-queue	Resets retry counters to zero and pushes tasks back to active priority list.	Click "Retry" in Dashboard or execute `pulse-cli retry <id>`.

Component	Purpose	Operator Action
Heartbeat Registry	Monitors connected worker hosts, active concurrency, and liveness.	Auto-prunes dead nodes after 30 seconds of missed heartbeats.

6. Empirical Performance Benchmarks

Evaluated under high-concurrency benchmarks on Linux x86_64 (13th Gen Intel Core i5-13450HX, 16 threads, local Redis instance):

Benchmark Operation	Throughput	Latency / Cost	Concurrency Mode
Atomic Enqueue (Parallel)	**112,400 tasks / sec**	8,897 ns / op	16 parallel goroutines + Redis Lua
Multi-Priority Dequeue	**85,000 tasks / sec**	11,700 ns / op	Strict priority check + in-flight tracking
Memory Footprint (Worker)	**< 25 MB RSS**	Zero allocation GC pauses	Static binary, minimal footprint

> **Summary & Resume Takeaway**
>
> PulseQueue demonstrates complete architectural mastery over distributed systems fundamentals: zero-loss crash recovery, atomic concurrency without distributed lock contention, self-healing background schedulers, and real-time operational observability.